**“Caixa Rápido”: processando pagamentos com interface e múltiplas estratégias**

**Contexto**

Você foi chamado para prototipar a parte de pagamentos de um ponto de venda. O sistema precisa aceitar **Pix**, **Cartão de Crédito** e **Boleto**. A ideia é escolher o meio de pagamento em tempo de execução, sem alterar o código que calcula o total.

**Objetivo didático**

- Definir uma **interface** que representa um processador de pagamento.
- Criar **múltiplas implementações** (Pix, Cartão, Boleto).
- Aplicar **polimorfismo** e **injeção por construtor** para independência do código cliente.
- (Opcional) Demonstrar **métodos default** na interface e **validação simples**.

**Requisitos funcionais**

1. Interface ProcessadorPagamento

```
public interface ProcessadorPagamento {
    boolean pagar(double valor);
    default void emitirRecibo(double valor) {
        System.out.println("Recibo: pagamento de R$ " +
String.format("%.2f", valor) + " confirmado.");
    }
}
```

2. **Implementações**
   - PagamentoPix
     - Construtor recebe chavePix (String).
     - pagar retorna true se valor > 0 e chavePix não está vazia; imprime no console que o Pix foi efetuado.
   - PagamentoCartaoCredito
     - Construtor recebe numero, nomeTitular, cvv.
     - pagar retorna true se os campos não estão vazios e valor ≤ limiteSimulado (ex.: 5.000); imprime autorização simulada.
   - PagamentoBoleto
     - Gera “linha digitável” simulada (pode ser um UUID) em pagar, **sempre retorna true** se valor > 0, e imprime instruções de pagamento.

3. **Classe Carrinho**

- Mantém uma lista de itens simples (String descricao, double preco).
- Método double total() soma os preços.
- Método finalizarCompra(ProcessadorPagamento proc) que:
  - Calcula valor = total().
  - Chama proc.pagar(valor).
  - Se true, chama proc.emitirRecibo(valor); caso contrário, imprime “Pagamento recusado”.

4. **Classe App (console)**

- Monta um carrinho com 3–4 itens de exemplo.
- Pede ao usuário escolher 1, 2 ou 3 para (1) Pix, (2) Cartão, (3) Boleto (pode usar Scanner, ou deixar pré-fixado se preferir).
- Cria a implementação correspondente e passa para finalizarCompra.

**Regras / restrições**

- Sem bibliotecas externas; tudo em console.
- O Carrinho **não pode** conhecer classes concretas de pagamento — apenas a **interface**.
- Validações podem ser simples (não precisa tratar exceções elaboradas).

**Guia de implementação (sugestão de roteiro)**

1. **Modelo dos Itens**

```
public class Item {
    private String descricao;
    private double preco;
    // construtor, getters
}
```

2. **Carrinho** com List<Item> (use ArrayList), adicionar(Item), total(), finalizarCompra(ProcessadorPagamento).

3. **Interface** ProcessadorPagamento + método default emitirRecibo.

4. **Implementações** PagamentoPix, PagamentoCartaoCredito, PagamentoBoleto.

5. **App**
   - Crie itens (ex.: "Café", 8.50; "Pão de queijo", 6.00; "Suco", 9.90).
   - Some tudo, peça o meio de pagamento, instancie a implementação.
   - Chame carrinho.finalizarCompra(processador).

6. **Testes manuais** alterando o meio de pagamento e valores (ex.: total > 5.000 deve falhar no cartão).

7.

**Exemplo esperado (console)**

```
Carrinho:
- Café: R$ 8,50
- Pão de queijo: R$ 6,00
- Suco: R$ 9,90
Total: R$ 24,40

Selecione o meio de pagamento [1-Pix, 2-Cartão, 3-Boleto]: 1
Processando PIX para chave: aluno@exemplo.com ...
Pagamento aprovado!
Recibo: pagamento de R$ 24,40 confirmado.
```